ASSIGNMENT.

INTELLIGENT EMERGENCY EVACUATION ROUTING SYSTEM.

CSA03112 - DATA STRUCTURE.

NAME : Daniel Jeswaji. T.
REG NO : 192565085.

Intelligent Emergency Evacuation Routing System

1. Problem understanding and formulation:

Problem:

The problem is to design an Intelligent Emergency routing system for a city containing more than 8000 intersections and 20,000 weighted roads. During disaster such as floods, earthquakes or industrial accident, road accidents conditions may change frequently.

Some roads may become blocked, while the travel cost of other road may increase due to congestion or damage. Therefore, the system must identify the safest and shortest available route from disaster locations to evacuation centres.

Expected outcome:

The system should:

=> Find the minimum-cost available route.
=> Avoid blocked roads.
=> Handle changing road weights
=> support repeated routing queries.
=> Identify unreachable evacuation centres.
=> use memory efficiently.

Available Data:

The system receives information about:

=> Intersection
=> Roads.
=> Travel cost.
=> Road status.
=> Disaster location.
=> Evacuation centres.

Assumptions:

=> Each intersection has unique vertex id.
=> road weights are non-negative.
=> A blocked road should not be used for routing.
=> updated road information is assumed to be correct.

constraints:

The system must handle more than 8000 vertices and 20,000 weighted roads. It should also support road changes, blocked roads, etc.

Intelligent Emergency Evacuation Routing System

=> Sutiable Graph ADT and Graph representation:-

The city road network can be represented using a weighted graph.

=> Vertices (V): Intersection.

=> Edges (E) : Roads.

=> Weight : Travel cost or travel time.

=> Blocked road : Disabled edge.

For this system, an Adjacency list is more sutiable than an Adjacency matrix.

The network has more than 8000 intersec-tions and only about 20,000 roads. An adjacency matrix would require VXV memory, which wastes a large amount of space. An adjacency list stores only the roads that actually exist.

=>Suitable C structures:-

```
typedef struct edge {
    int destination;
    int weight;
    int active;
    struct edge *next;
} edge;
typedef struct {
    edge *head;
} vertex;
typedef struct {
    int vertex;
    int distance;
} Heapnode;
typedef struct {
    HeapNode Heap[10000];
    int size;
} Minheap;
typedef struct {
    int source;
    int destination;
    int cost;
} Route;
```

Here, active = 1 represents an available road and active = 0 represents a blocked road.

Hashset for processed Intersections:-

A Hashset is used to store intersection that have already be processed.

```
Processed[u] = 1;
Before processing a vertex;
If (Processed[u])
    continue;
```

Rate reconstruction:

A parent[] array is used to store the Previous vertex.

```
Parent[v] = u;
```

After reaching the destination, we move backwards.

Destination ⟶ Parent ⟶ Parent → source.

Example:

0 → 2 → 4 → 6 → 7.

=> Dynamic road weight and Road Status Update:

When a disaster occurs, the road weight or road status can change.

For example:

```
edge->weight = newWeight;
edge-> active = status;,
```

If a road is blocked, its status is set to 0. Dijkstra's algorithm will ignore that road.

If the weight changes, the new value is used for next route calculation.

Use of min-Heap in Dijkstra's Algorithm:-

A Min-Heap is used to select the intersection with the minimum travel cost.

Steps:-

=> Insert the source vertex with distance 0.

=> Extract the vertex with minimum distance.

=> check all neighbouring roads.

=> Update the distance if a shorter Path is found.

=> Insert or Update the vertex in the Min-Heap.

=> Repeat until the destination is reached

use of Modern tools:-

⇒ The proposed intelligent Emergency Evacuation routing system can be implemented using the C Programming language. A suitable programming environment such as IDE or C compiler can be used to write, compile and test the program.

⇒ The program can be tested by creating different road network condition, such as normal roads, blocked roads and changes in travel costs. Debugging tools available in the Programming environment can help identify errors and verify whether the sortest route is calculated correctly.

⇒ A version-control platform can also be used to maintain the source code and track changes made during development. These tools help in testing, improving and maintaining the Proposed system.

Result and validation:-

⇒ The proposed system is validated by testing it under different road conditions. In a normal road network, the system should identify the minimum-cost route between the source and the evacuation centre.

⇒ when a road is blocked, the algori-thm ignore that road and searches for another available route. if the travel cost of road changes, the system recalculates the route based on the updated weight.

⇒ overall, the validation shows that the proposed system can respond to changing road conditions and provide an efficient evacuation route.

Output

```
==========================================
 INTELLIGENT EMERGENCY EVACUATION ROUTING SYSTEM
==========================================

City Intersections: 8
Source Intersection: 0
Destination Intersection: 7

---------- NORMAL CONDITION ----------

Result: Shortest route found successfully!
Evacuation Route : 0 -> 2 -> 4 -> 6 -> 7
Total Travel Cost : 16

---------- AFTER BLOCKING ROAD (2 -> 4) ----------
Road (2 -> 4) is now BLOCKED.

Result: Shortest route found successfully!
Evacuation Route : 0 -> 2 -> 5 -> 6 -> 7
Total Travel Cost : 17

---------- AFTER CHANGING ROAD WEIGHT (1 -> 3) ----------
Road (1 -> 3) weight updated to 10.

Result: Shortest route found successfully!
Evacuation Route : 0 -> 2 -> 5 -> 6 -> 7
Total Travel Cost : 17

---------- ROUTE CACHE TEST (Repeat Query) ----------

Route found in Cache! (No Dijkstra run again)
Source: 0
Destination: 7
Cached Travel Cost: 17

---------- UNREACHABLE DESTINATION TEST ----------

Result: Destination is unreachable from the given source!
```

Route caching:-

Repeated evacuation queries may request the same route.

For example:

source = 0

Destination = 7.

The calculated route can be stored in cache

Handling special cases:-

Invalid vertex:

```
if (source <0 || destination >= vertices)
    Printf ("Invalid vertex");
```

Duplicate road:

Before adding a road, check whether it already exists.

Blocked Road:

```
if (edge -> active == 0)
    continue;
```

Unreachable Destination:

```
if (distance [destination] == INF)
    Printf ("Evacuation centre unreachable");
```

=> Dijkstra's Algorithm vs Floyd-Warshall:

Feature	Dijkstra	Floyd-Warshall
Purpose.	single-source shortest Path	All-pairs shortest Path.
Time complexity	$O((V+E)\log V))$	$O(V^3)$
Memory.	$O(V+E)$ with adjacency list	$O(V^2)$
Large network	Suitable	Not suitable.
Dynamic updates	Better	Expensive.

From this problem, Dijkstra's algorithm is the better choice because the network is large and road conditions change frequently.

complexity Analysis:

Time complexity :

$$O((V+E)\log V)$$

space complexity:

$$O(V+E)$$

⇒ vertices > 8000
⇒ roads ≈ 20,000

=> can the system meet the 200 ms requirement:

=> Adjacency list reduce memory storage.

=> Min-Heap Quickly select the minimum-distance vertex.

=> Hashset avoids repeated processing.

=> Blocked roads are skipped.

=> Route caching speed up repeated Queries.

Therefore, the system should be tested using large input data before confirming the requirement.

=> when 30% of roads become unavailable:

If 30% of 20,000 roads become unavailable

30% of 20,000 = 6,000

Therefore;

=> 6000 roads become blocked.

=> 14,000 roads remain available.

Possible situations are:

=> Travel cost may increase because of longer routes.

=> Some evacuation centres may become unreachable.

=> Cached routes must be updated or invalidated.

conclusion:-

The Intelligent emergency evacuation routing system uses an adjacencyList, Min-Heap, HashSet, and Dijkstra's algorithm to efficiently find the shortest path. therefore, this design is suitable for managing emergency routing in a large smart-city network.

Broader considerations:-

=> Proposed system can improve public safety by helping people find available evacuation routes during disasters.

=> It supports smart-city infrastructure and emergency management. During climate-related disasters such as floods, dynamic route selection can reduce delays.

=> Accurate road information because incorrect routing information during an emergency could affect public safety.

Student reflection:-

Through this assignment I understood how data structures can be used to solve a real-world emergency problem. I learned how graphs represent road networks and how an adjacency list reduces memory usage. I also understood how a min-Heap improves the performance of Dijkstra's algorithm.

References.

=> Data Structures course note and classroom materials.

=> Jayalakshmi, P., & Manimekalai, K. (2025). Unleashing the power of graph theory in Data structures.

=> Diestel, R. (2025). Graph Theory. Springer.

=> Zeil, S. J. (2025). Graph: Sample Algorithms and Dijkstra's Algorithm. old Dominion University.